PicoCalc LLVM World Lab: Host-Side Freestanding RV32 Execution

Architecture and implementation report

28 October 2026

Contents

Abstract **1**

1. Relationship to the existing PicoCalc OS **2**

2. Compiler and execution boundaries **2**

3. Why a restricted JavaScript frontend? **2**

4. A freestanding LLVM toolchain **3**
4.1 Why two fixed code slots? 3

5. ABI, state and memory permissions **3**

6. Transactional world turns **4**

7. Hot reload and the generation protocol **4**

8. A small framebuffer demonstration **5**

9. Validation **6**

10. Unimplemented components and hardware acceptance gates **6**

11. Suggested next two milestones **6**

12. Primary references **7**

13. Lab exercises **7**

Conclusion **7**

Abstract

This proof of concept explores an LLVM-powered PicoCalc operating system whose programs are compiled *outside* the device into freestanding 32-bit RISC-V native code, loaded dynamically, and replaced while preserving state. Two completely separate host processes are involved: a `jitd` LLVM/Clang compiler service and a `pico-sim` target simulator written in C++17. The compiler accepts a small integer-oriented subset of JavaScript or trusted freestanding C. It emits actual RV32IM instructions inside ELF32 files. The simulator loads and executes those instructions with private state and stack regions, a small `ecall` capability ABI, bounded instruction budgets, a transactional effect log, schema migration, and generation-based hot reload.

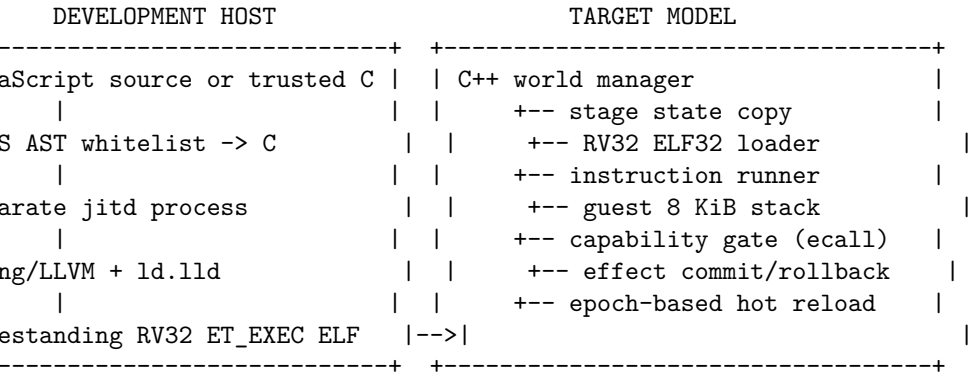
Important limits: This is not a full ECMAScript engine, an LLVM ORC runtime, a secure user-mode sandbox, or an ESP32-P4 emulator. The guest has no libc or OS; the host simulation program uses standard C++ libraries. The proof validates the *architecture seam* of a remote JIT and a freestanding native runtime before porting the design to physical hardware.

1. Relationship to the existing PicoCalc OS

The supplied `picocalc-os-src(2).zip` describes a C nucleus with supervised JavaScript *worlds*: drivers, a compositor, and applications. Each original world has its own QuickJS heap and execution budget; worlds exchange messages and acquire hardware through revocable capabilities. The original simulator models an STM32 keyboard, the ST7365P panel, bus timing, a virtual clock, and reproducible screen captures. It targets a PicoCalc fitted with an ESP32-P4 replacement module.

The present experiment intentionally implements **one** world and a minimal kernel rather than copying the complete operating system. It retains the relevant behavioral contracts: an event turn, a committed state separate from executing code, capability-mediated host effects, rollback on fault, epochs on queued events, and hot reload at an execution boundary. This isolates the main research question: can target-format native code be delivered and safely replaced without embedding LLVM or libc inside the guest?

2. Compiler and execution boundaries



The ESP32-P4 has RV32IMAFC high-performance CPU cores. This PoC uses the **RV32IM subset** (`-march=rv32im -mabi=ilp32`) so its interpreter can stay small; the generated instructions are compatible with the larger CPU ISA. The simulator supports integer arithmetic, multiply/divide, loads/stores, branches, jumps, function calls and a deliberately limited `ecall` convention. It does **not** model FPU operations, compressed instructions, atomics, cache hierarchy, interrupt controllers, PMP, or peripheral timing.

Actual physical ESP32-P4 execution would remove this interpreter and execute native code from hardware-verified executable memory. The addresses in this prototype are **synthetic simulation addresses**, not established firmware mappings.

3. Why a restricted JavaScript frontend?

A correct JavaScript engine needs objects, strings, IEEE-754 binary64 Numbers, coercions, exceptions, Promise jobs, garbage collection, and a shared representation of optimized/interpreted frames. LLVM does not provide those semantics. To test the *compiler/runtime boundary* without claiming to have ported Static Hermes, the initial frontend only handles a statically checked integer domain.

Example `worlds/counter_v1.js`:

```
export const schema = 1;
export function step(state, event, arg) {
  if (event === 1) {
    state.score = (state.score + arg) | 0;
    state.ticks = (state.ticks + 1) | 0;
    cap.emit(state.score);
  }
  return 0;
}
```

The host TypeScript parser constructs an AST. `compiler/js2c.js` walks a whitelist of expressions, statements, fixed `PicoState` fields, and kernel capabilities. It rejects arbitrary property access, `eval`, arbitrary calls, strings, classes, async functions, and unsupported operators. It generates a freestanding C entrypoint:

```
int32_t pico_world_step(PicoState *state, uint32_t event, int32_t arg) {
    if (event == 1) {
        state->score = (state->score + arg) | 0;
        state->ticks = (state->ticks + 1) | 0;
        pico_cap_emit(state->score);
    }
    return 0;
}
```

The C compiler uses `-fwrapv` to give signed addition, subtraction and multiplication defined wrapping behavior. The tests compare bounded inputs against Node.js for the implemented subset. **The resulting integer semantics are not equivalent to arbitrary JavaScript Number operations.** In particular, division and left shifts are excluded instead of silently producing C-specific behavior.

There is also a native C fixture with a `noinline` helper. It exercises a real RISC-V function call, stack frame and return without any JavaScript frontend. This demonstrates that the module ABI does not depend on the language used to construct the code.

4. A freestanding LLVM toolchain

`compiler/compile.py` invokes Clang and LLD with explicit target constraints:

```
clang --target=riscv32-unknown-elf \
  -march=rv32im -mabi=ilp32 -O2 \
  -nostdlib -ffreestanding -fno-builtin \
  -fno-stack-protector -fno-pic -mno-relax \
  -fwrapv -fno-unwind-tables \
  -c module.c -o module.o

ld.lld -m elf32lriscv --no-undefined --no-relax \
  -T generated-link.ld module.o -o module.elf
```

No guest C runtime is linked, no `main` or `_start` is required, and there are no unresolved imports. The entry is an actual callable function `pico_world_step`. Cross-compiled images are ET_EXEC ELF32, little-endian, machine RISC-V. `readelf -d` reports no dynamic dependencies, and program headers contain RX code segments, not writable-and-executable segments.

The compiler daemon (`compiler/jitd.py`) communicates through a Unix-domain socket with JSON-line requests containing source, destination, optimization level and code slot. It runs as a separate process from the world simulator. The output path is restricted to an assigned development directory. This is a **trusted local developer service**, not an authenticated or sandboxed remote compiler.

4.1 Why two fixed code slots?

For the first experiment, LLVM prelinks code at `0x00100000` or `0x00200000`. The world simulator only loads the ELF; it does not relocate it. A new generation is compiled for whichever slot is inactive. This is a deliberately simple form of **remote ahead-of-time compilation on demand**, akin to a remote JIT, not in-device LLVM or ORC code generation. The next module-loader milestone is to introduce relocations or PIC and an authenticated manifest instead of hard-coded simulation addresses.

5. ABI, state and memory permissions

`include/pico_abi.h` defines the entire guest-visible state:

```c
typedef struct PicoState {
    uint32_t schema;
    int32_t score;
    int32_t ticks;
    int32_t flags;
} PicoState;
```

Guest location	Role
0x00100000 / 0x00200000	RX compiled-code slot
0x30000000	RW 16-byte tentative world state
0x80000000	RW guest stack region of 8 KiB
a0	Pointer to state
a1 / a2	Event or version and argument
sp / ra	Isolated guest stack and return sentinel
a7	Capability identifier at ecall

The ELF32 loader checks architecture, segment bounds, RWX combinations and export symbols. The RV32 interpreter implements its own read/write/execute checks at every guest memory access. This checks the prototype's ABI invariants but is not equivalent to real RISC-V PMP isolation. A malicious native program running with firmware machine-mode privilege could simply ignore API conventions; the actual system needs hardware user-mode/PMP protection or an equivalent mechanism.

The freestanding guest's `pico_cap_emit` function uses inline RISC-V assembly:

```c
register int32_t a0 __asm__("a0") = value;
register int32_t a7 __asm__("a7") = PICO_CAP_EMIT;
__asm__ volatile("ecall" : "+r"(a0) : "r"(a7) : "memory");
```

There is no POSIX `write`, console, allocator, or dynamic-symbol lookup in the guest. The simulator traps `ecall`, checks the world capability, and queues the proposed effect for later commit. Capabilities include `emit`, `display` (validated rectangle drawing), and explicit `abort`. The kernel refuses unauthorized effects and all effects during state migration.

6. Transactional world turns

A turn executes with a copy of the committed `PicoState`, an isolated stack, and an empty effect log. It either finishes and atomically commits the updated state plus buffered effects, or faults and discards both.

```
committed state --clone--> staged state --RV32 turn--> success? --yes--> commit state + effects
                                                                \\--no----> discard state + effects
```

Fault causes include an illegal or unsupported RV32 instruction, invalid memory access, capability denial, explicit `abort`, or instruction-fuel exhaustion. Every fetched RV32 instruction consumes one fuel unit. This provides deterministic bounded execution on the host model; hardware implementation would need a timer interrupt, instrumented native fuel checks, or another preemption mechanism.

This is a simplified version of the original PicoCalc OS's deferred-effect rule. Physical I/O cannot always be rolled back after being transmitted. Real drivers should execute effects only after commit or implement explicit compensation semantics.

7. Hot reload and the generation protocol

Each world has an active module, persistent state, and an epoch. Reload is a **shadow operation**: the kernel loads the new ELF in the inactive code slot, copies committed state, invokes `pico_world_migrate(state, old_schema)` in the shadow context, and checks that the return value is zero and the migrated schema matches the manifest. Migration has no effect permissions.

If migration succeeds, the kernel publishes the new state, module pointer and incremented epoch. Any queued event from an older epoch is rejected. If migration fails, the old code and state remain active. Because this host simulator has one execution lane, reload occurs between turns; a real multicore implementation must delay retiring code until no guest return address points into it.

Recorded integrated trace:

```
epoch 1 / v1: add(3)              -> score 3
queue add(7), epoch 1
reload v2 / schema 2              -> score 3, flags 42, epoch 2
old queued add(7)                -> rejected as stale
v2: add(4)                       -> score 43 (v2 computes +arg*10)
try v3: migration calls abort    -> rejected; v2 stays active
v2: add(1)                       -> score 53
```

`scripts/watch.py` monitors a source file and sends its revisions to `jitd`. It reads each reload acknowledgement. A rejected reload does *not* advance the active-code-slot model; the watcher can then retry the correct inactive slot after the next edit.

8. A small framebuffer demonstration

The simulator includes a 320×320 RGB565 framebuffer, but not the original ST7365P bus model. The compiled guest draws through `cap.rect(x,y,w,h,rgb565)`; the kernel checks bounds and stages rectangle effects. When the turn commits, these are applied to the panel. A test stages a black full-screen fill and then calls `cap.abort(99)`: the effect is discarded. The before/after PPM images have the same SHA-256 hash.



Figure 1: The simulated panel generated by RV32 code, not physical hardware

The screenshot illustrates the division of authority: applications can *request* drawing, while only the kernel modifies the committed framebuffer. A full PicoCalc port would retain the existing compositor's declarative UI trees and

execute the ST7365P/SPI driver after commit.

9. Validation

The repository includes integration tests of ELF architecture and no-libc linkage, native arithmetic and function calls, state preservation, rollback, permission checks, schema mismatch handling, stale messages, infinite loops, and rendering. Two source versions are additionally compared against Node.js over 80 deterministic input pairs each, checking score, tick count and emitted values. LLVM `-O0`, `-O2`, and `-Oz` outputs are tested against the same expected world behavior. The restricted subset passes those comparisons; this is not test262 evidence.

A representative Linux ASan+UBSan build of the **host simulator** also passed a scenario containing native-code execution and migration. It does not address unsafe memory behavior in physical native execution. No target-cycle performance claims are made: measuring an RV32 instruction interpreter hosted on x86-64 is not measuring an ESP32-P4. The prototype records instruction counts and generated-code sizes instead.

10. Unimplemented components and hardware acceptance gates

Area	This prototype	Next required capability
JavaScript runtime	Typed i32 syntax subset	Complete JS semantics, interpreter, GC, frame states, deoptimization
Compiler	External Clang/LLVM on source update	Hotness feedback, semantic IR, ORC/object lifecycle if useful
Loader	Two fixed ELF32 slots	Arbitrary relocatable/PIC objects, manifest, signatures, versioned code cache
Execution	Host RV32IM interpreter	Direct native P4 RV32IMAFC execution with trap-based protection
Memory	Simulated RX/RW segment checks	Verified executable IRAM or PSRAM/flash mapping, PMP, cache sync
OS	One stateful world and event lane	All driver/application worlds, supervisor, real I/O and async job queues
Display	Kernel RGB565 framebuffer	Original ST7365P, SPI DMA, compositor, keyboard and virtual-clock parity

Before loading native code on the ESP32-P4, verify real executable memory mappings and privileges in the selected ESP-IDF build, synchronize modified code through the required instruction cache APIs, implement an appropriate protected trap boundary, and coordinate both CPU cores. The documented `CONFIG_SPIRAM_XIP_FROM_PSRAM` moves certain firmware sections to PSRAM; it **does not by itself prove that arbitrary runtime PSRAM buffers are executable**. This requires a physical-device test.

11. Suggested next two milestones

N1 — Native relocation and authentic hardware addressing. Replace hardcoded code slots with a versioned module container, relative relocations, a small authorized import table, integrity checks, code/stack size declarations, and explicit executable memory publication/retirement. Test the loader with two independent load addresses and with a real RV32 environment such as QEMU or hardware.

N2 — Actual JS runtime with native tier. Investigate Static Hermes or another portable JS runtime, keep the existing PicoCalc `app().state().on().screen()` user API, define tagged values and GC barriers, and compile selected hot bytecode through LLVM on the host. Add interpreter fallback and frame-state recovery. Do not treat translating a JS AST to C integers as a substitute for complete JavaScript semantics.

12. Primary references

1. Original project: user-supplied `picocalc-os-src(2).zip` (`README.md`, `nucleus/kernel.c`, `nucleus/vm.c`, `js/prelude.js`, `js/system.json`).
2. Espressif, *ESP32-P4 Series Datasheet*: https://documentation.espressif.com/esp32-p4_datasheet_en.html.
3. ESP-IDF, *ESP32-P4 External RAM and PSRAM XiP*: https://docs.espressif.com/projects/esp-idf/en/stable/esp32p4/api-guides/external-ram.html.
4. ESP-IDF, *Memory Synchronization*: https://docs.espressif.com/projects/esp-idf/en/stable/esp32p4/api-reference/system/mm_sync.html.
5. *RISC-V ELF psABI*: https://github.com/riscv-non-isa/riscv-elf-psabi-doc.
6. LLVM, *RISC-V backend*: https://llvm.org/docs/RISCVUsage.html.
7. LLVM, *ORC JIT design*: https://llvm.org/docs/ORCv2.html.
8. *RISC-V unprivileged ISA*: https://github.com/riscv/riscv-isa-manual.
9. Xu and Kjolstad, *Copy-and-Patch Compilation*, 2021: https://compilers.stanford.edu/publications/copy-and-patch/.
10. Meta, *Hermes/Static Hermes source*: https://github.com/facebook/hermes.

13. Lab exercises

1. Add a third capability with runtime rights checks, and show rollback of a turn that invokes it before aborting.
2. Implement an ELF relocation subset and prove the same compiled function executes at two different addresses.
3. Extend the simulator to RV32IMC and compare instruction counts under `-march=rv32imc`.
4. Replace the fixed 16-byte state with a bounded serializable schema; migrate it across three versions and deliberately fail the fourth.
5. Test `-O0`, `-O2`, and `-Oz` compiler output, comparing compilation time, machine-code size and interpreted *instruction counts* (not target-clock throughput).
6. On the ESP32-P4, demonstrate a signed, read-only machine-code image executed from a verified executable mapping, then retired after a safe point.

Conclusion

A host-side compiler/target split is viable. The prototype executes real freestanding RV32 code produced by LLVM, preserves per-world state through versioned hot reload, provides a minimal capability trap ABI, and commits hardware-like effects only after successful turns. Its limitations are explicit: the JS frontend is a restricted DSL; executable memory and privileges are simulated, not verified on the ESP32-P4. The next meaningful step is a real relocatable loader and physical executable-memory experiment, followed by integration with a complete JavaScript runtime.